Decode account data

Decode raw Solana account bytes into real, readable data and see what's inside.

CHALLENGE

Decode account data

The Scenario

Yesterday you built an account explorer that could pull up basic account info from the Solana network. You saw fields like `lamports`, `owner`, and `data`. But that `data` field? It was just a blob of raw bytes. A wall of base64 gibberish.

Think of it like opening a database record and seeing a binary column. You know something meaningful is in there, but without the schema, it's just noise. Today, you're going to crack that open. You'll take raw on-chain bytes and turn them into structured, human-readable information using a serialization format called Borsh.

The Challenge

What You'll Need

- A terminal with Node.js (v18+) installed
- A code editor (VS Code, Cursor, or similar)
- Your project from Day 23, or a fresh directory
- A Solana devnet or mainnet RPC endpoint (the public `https://api.devnet.solana.com` works fine)

Steps

1. Create a new project directory and initialize it using this Gist.
2. Create a file called `decode.mjs`. Start by connecting to the network and fetching the raw account data for Wrapped SOL, a well-known SPL Token mint on mainnet. Use this Gist for decode.mjs.

Run this and look at what comes back. You'll see the owner is the Token Program, the data is 82 bytes long, and the hex dump is completely unreadable. That's the problem you're solving.

3. Now decode those bytes using the `Mint` codec from `@solana-program/token`. This codec knows exactly how the Token Program structures its mint accounts, and gives you a structured object in one line. Add the contents of this Gist to your file:

The `__option` discriminator is how `@solana/kit` represents Rust's `Option` in JavaScript: either `{ __option: "Some", value: ... }` or `{ __option: "None" }`. No null checks, no truthy guesswork. The shape tells you which arm you're in.

This is the **ergonomic path**. One decoder call and you're done. In real code, this is what you'd reach for. The next step does the exact same work by hand, byte by byte, so you can see what `getMintDecoder()` is actually doing under the hood.

4. That was the easy path, using a pre-built codec. Now try it the manual way so you understand what's actually happening at the byte level. A Mint account's 82 bytes are structured like this:
    - Bytes 0-3: `mintAuthorityOption` (u32, 1 = present, 0 = none)
    - Bytes 4-35: `mintAuthority` (32-byte public key)
    - Bytes 36-43: `supply` (u64, little-endian)
    - Byte 44: `decimals` (u8)
    - Byte 45: `isInitialized` (boolean)
    - Bytes 46-49: `freezeAuthorityOption` (u32)
    - Bytes 50-81: `freezeAuthority` (32-byte public key)

Add a manual decoding section to prove this to yourself, using this Gist

A few things worth understanding here:

What is `DataView`? A `Uint8Array` only lets you read **one byte at a time**. But `supply` is 8 bytes and `mintAuthorityOption` is 4 bytes, so you need to read several adjacent bytes and combine them into a single number. `DataView` is the standard JavaScript API for exactly that. It's a thin wrapper around the same underlying bytes that exposes methods like `getUint8`, `getUint32`, and `getBigUint64`. Nothing Solana-specific. You'd use it to parse any binary format.

Why the `true` second argument? That's the **little-endian** flag. `DataView` defaults to *big-endian* (most significant byte first, the way humans write numbers), but Solana stores everything **little-endian** (least significant byte first). So the bytes `[0x05, 0x00, 0x00, 0x00]` mean `5` in Solana's world, not `83886080`. Forgetting that flag is the single most common decoding bug you'll hit. Every multi-byte read on Solana data needs `true` here.

Why `getBase58Decoder` for the address? A Solana address isn't a special data type. It's just **base58-encoded 32 bytes**. You've already used `getBase64Encoder` (base64 string → bytes) and `getBase16Decoder` (bytes → hex string) above. `getBase58Decoder` is the third member of the same family: bytes → base58 string. That string *is* the address you'd see on Solana Explorer.

5. Finally, compare your manual work against Solana's built-in JSON parser. The RPC has a `jsonParsed` encoding that does the decoding server-side for known programs, using this Gist

Your manual numbers should match exactly. If they do, you've proven you understand what the RPC is doing under the hood.

Run It

```
node decode.mjs
```

You should see three sections of output: the codec decode, your manual byte-level decode, and the RPC's JSON-parsed version. All three should agree on the supply, decimals, and authority fields.

What Just Happened

Every Solana account stores its state as a flat array of bytes. There's no built-in schema, no column names, no JSON structure. The program that owns the account defines how those bytes should be interpreted, and the standard format is called Borsh (Binary Object Representation Serializer for Hashing). Borsh lays out fields contiguously in declaration order, uses little-endian encoding for numbers, and adds a 4-byte length prefix for variable-length types like strings and vectors. It's compact, deterministic, and fast.

If you've worked with protocol buffers, MessagePack, or even binary file formats in Web2, this is the same idea. The difference is that on Solana, the schema lives in the program's source code, not alongside the data. That's why you need a codec definition (like `getMintDecoder()`) or the byte-level specification to make sense of what you're reading. The `jsonParsed` encoding is a convenience the RPC provides for well-known programs, but for custom programs you'll need to bring your own decoder.

`@solana/kit` leans into this hard. Instead of hiding serialization behind monolithic helper classes, it exposes codecs as composable building blocks: `getU64Decoder()`, `getAddressDecoder()`, `getStructDecoder()`, `getOptionDecoder()`, and so on. Program-specific packages like `@solana-program/token` are just pre-assembled codecs over the same primitives. Once you see the pattern, you can write a decoder for any program, including the ones you'll write yourself, without reaching for a heavy SDK.

You also met three text encodings today, and they're worth keeping straight because they all show up constantly in Solana code. Each one is a different **way to spell the same bytes** as a string. They don't change the bytes, only how you write them down. **Base64** is what the RPC sends account data as, because it's compact (~33% overhead). **Base16** (hex) is what you reach for when debugging, because each byte is exactly two characters and easy to count. **Base58** is what addresses are written in. It deliberately leaves out characters that look alike (the digit zero vs capital O, lowercase L vs capital I), so addresses are harder to misread or mistype when humans copy them by hand. A Solana address isn't a special type; it's just the base58 spelling of a 32-byte Ed25519 public key. That's why `getBase58Decoder().decode(authorityBytes)` produces the same string you'd see on Solana Explorer.

This skill becomes essential as you start building on Solana. When you write your own programs, you'll define structs in Rust that Borsh serializes into these same byte arrays. When you build frontends, you'll deserialize them back. Today you proved you can do that translation by hand, which means you'll never be stuck staring at raw bytes wondering what they mean.

Resources

- Solana Kit Documentation
- Solana Docs: Account Model
- Solana Docs: getAccountInfo RPC Method
- Borsh: Binary Object Representation Serializer for Hashing
- SPL Token Program Documentation

Submission

Take a screenshot of your terminal showing all three decoded outputs (codec, manual, and jsonParsed) matching each other. Submit it here.

Submit your project →